

系统开发工具基础第二周实验报告

涂睿昕—25020007115

2026 年 9 月 11 日

1 实验概述

本次实验为系统开发工具基础第二周课上实验,实验环境为 VirtualBox 中的 Ubuntu 虚拟机。本次完成四项课上实验:进程与信号控制、IDE 语义重构开发反馈、调试器定位程序缺陷、性能分析与程序优化。实验主要练习 Linux 进程与信号、Shell 任务调度、Python 调试器、性能剖析工具、cgroups 资源隔离、ulimit 资源软限制,掌握多终端协同观测、内核资源管控与命令行性能排查的基本流程。

2 课上实验题目

2.1 q05: 控制一个可清理的后台任务

2.1.1 题目要求

编写 worker.sh 脚本,实现信号捕获逻辑;前台启动脚本,将标准输出、标准错误分别重定向至日志文件;使用 Ctrl-Z 挂起任务,再使用 bg 将任务转入后台继续运行;通过 jobs -p 获取进程 PID,禁止手工抄写 PID;向进程发送 SIGTERM 信号,禁止使用 kill -9;验证 cleanup.log 中输出 CLEAN_EXIT,确认任务正常退出。

2.1.2 实现过程与关键命令

```
cd ~/sdt-course/week2/q05
cat > worker.sh <<'EOF'
#!/usr/bin/env bash
trap 'echo CLEAN_EXIT >> cleanup.log; exit 0' TERM INT
n=0
while true; do
echo "$n"
n=$((n+1))
```

```
sleep 1
done
EOF
chmod +x worker.sh

./worker.sh > stdout.log 2> stderr.log
# Ctrl-Z 挂起任务
bg
jobs
jobs -p
kill -TERM 10098
jobs
cat cleanup.log
```

2.1.3 运行结果

执行结束后，`cleanup.log` 文件内存在字符串 `CLEAN_EXIT`，进程收到信号后正常退出，无进程残留；`stdout.log` 持续输出递增的数字序列，`stderr.log` 为空。

```
^Z
[1]+ 已停止                  ./worker.sh > stdout.log 2> stderr.log

[1]+ ./worker.sh > stdout.log 2> stderr.log &

[1]+ 运行中                  ./worker.sh > stdout.log 2> stderr.log &

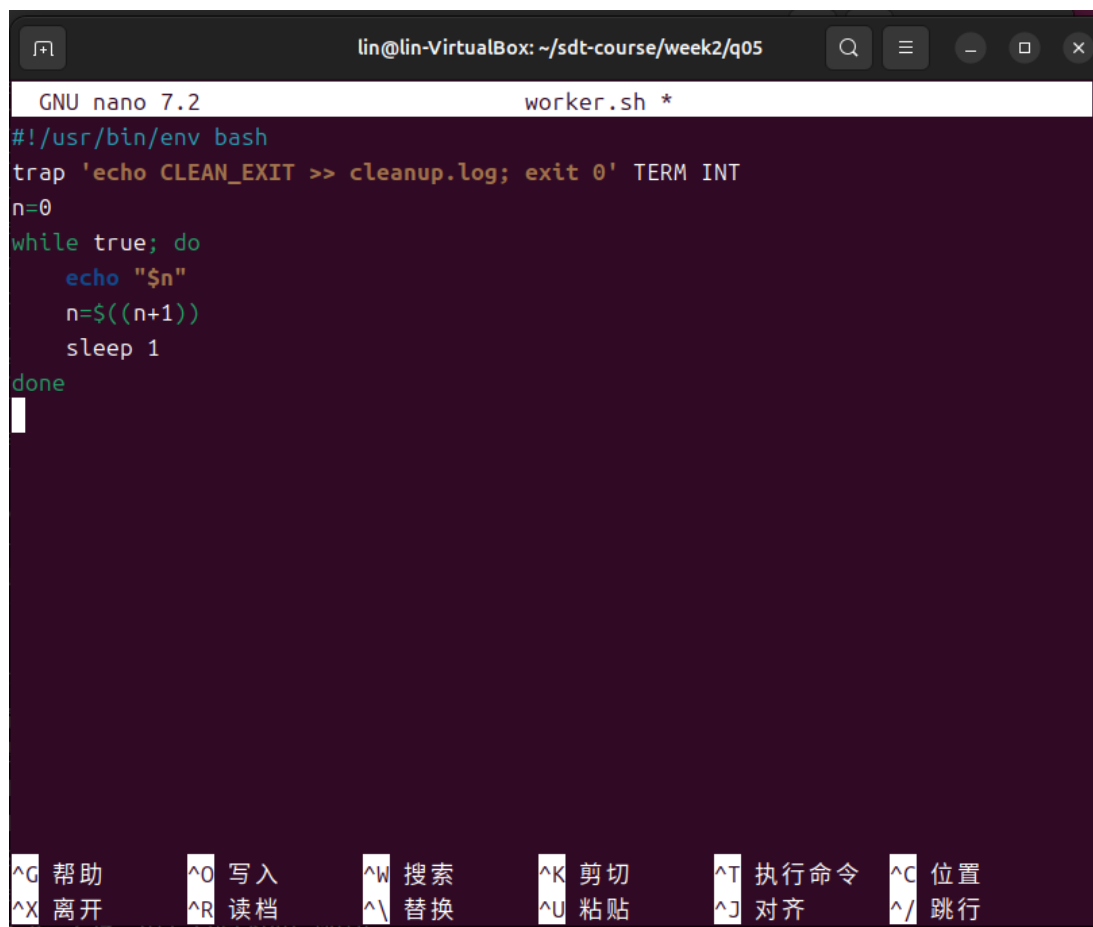
10098

[1]+ 已完成                  ./worker.sh > stdout.log 2> stderr.log

CLEAN_EXIT
```

2.1.4 遇到的问题与解决

- 问题 1：使用 `kill -9` 强制杀死进程，`trap` 无法捕获该信号，`cleanup.log` 不会输出清理标记。解决：直接使用 `kill` 发送 `SIGTERM`，给予脚本执行 `trap` 定义的清理逻辑的机会。
- 问题 2：手动复制粘贴 PID，不符合题目不手工抄写 PID 的约束。解决：使用 `jobs -p` 获取后台任务 PID 并赋值给 `shell` 变量。



```
lin@lin-VirtualBox: ~/sdt-course/week2/q05
GNU nano 7.2 worker.sh *
#!/usr/bin/env bash
trap 'echo CLEAN_EXIT >> cleanup.log; exit 0' TERM INT
n=0
while true; do
    echo "$n"
    n=$((n+1))
    sleep 1
done
```

^G 帮助 ^O 写入 ^W 搜索 ^K 剪切 ^T 执行命令 ^C 位置
^X 离开 ^R 读档 ^\ 替换 ^U 粘贴 ^J 对齐 ^/ 跳行

图 1: q05 worker.sh 截图

2.2 q06: 语义重构与本地开发反馈

2.2.1 题目要求

创建 `math_utils.py`、`app.py`、`test_math_utils.py` 三份源码；借助 Python 语言服务器实现跳转到定义、查找引用；使用重命名符号功能将 `total_price` 同步修改为 `calculate_total`；临时加入未使用的 `import` 语句，使用 `ruff` 检测并删除无用导入；执行 `ruff` 检查格式化与 `pytest` 单元测试，保证全部通过。

2.2.2 实现过程与关键命令

```
cd ~/sdt-course/week2/q06
nano # 创建三份python源码文件
code .
# VS Code语言服务器：跳转定义（F12）、查找所有引用（shift+F12）
# 使用重命名符号完成标识符全局替换（F2）
# 临时添加import os复现未使用导入告警
ruff check .
ruff check --fix
ruff check .
pytest test_math_utils.py -v
```

2.2.3 运行结果

重命名符号操作后，定义与全部引用位置同步更新；`ruff` 无告警输出；`pytest` 全部测试用例执行 **PASS**。

```
I001 [*] Import block is un-sorted or un-formatted
--> app.py:1:1
1  / import os
2  from math_utils import calculate_total
    ^
3
4  print(calculate_total(12.5, 4))

help: Organize imports
1  import os
2
3  from math_utils import calculate_total

F401 [*] `os` imported but unused
--> app.py:1:8
1  import os
    ^^
```

```

2   from math_utils import calculate_total

help: Remove unused import: `os`
- import os
1   from math_utils import calculate_total

I001 [*] Import block is un-sorted or un-formatted
--> test_math_utils.py:1:1
1   from math_utils import calculate_total
                                ^^

2
3   def test_total_price():

help: Organize imports
2
3
4   def test_total_price():

Found 3 errors.
[*] 3 fixable with the `--fix` option.

Found 2 errors (2 fixed, 0 remaining).

All checks passed!

===== test session starts =====
platform linux -- Python 3.12.3, pytest-9.1.1, pluggy-1.6.0
rootdir: /home/lin/sdt-course/week2/q06
collected 1 item

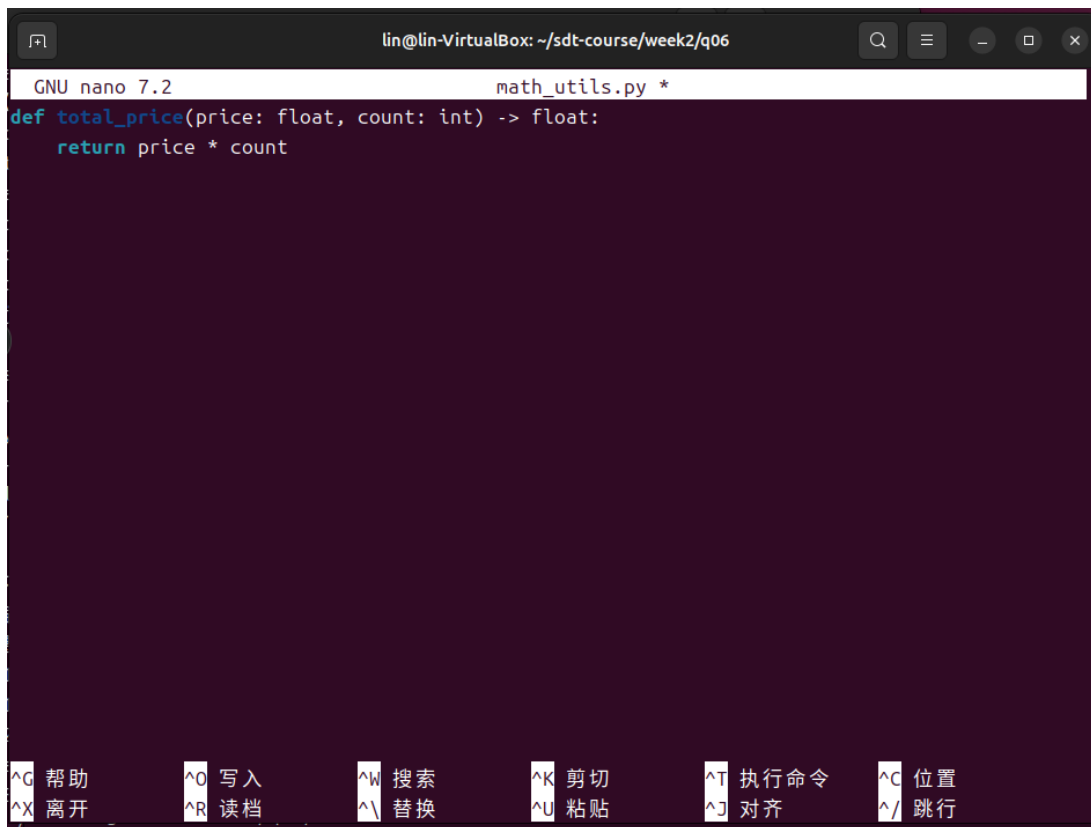
test_math_utils.py .
    [100%]

===== 1 passed in 0.00s =====

```

2.2.4 遇到的问题与解决

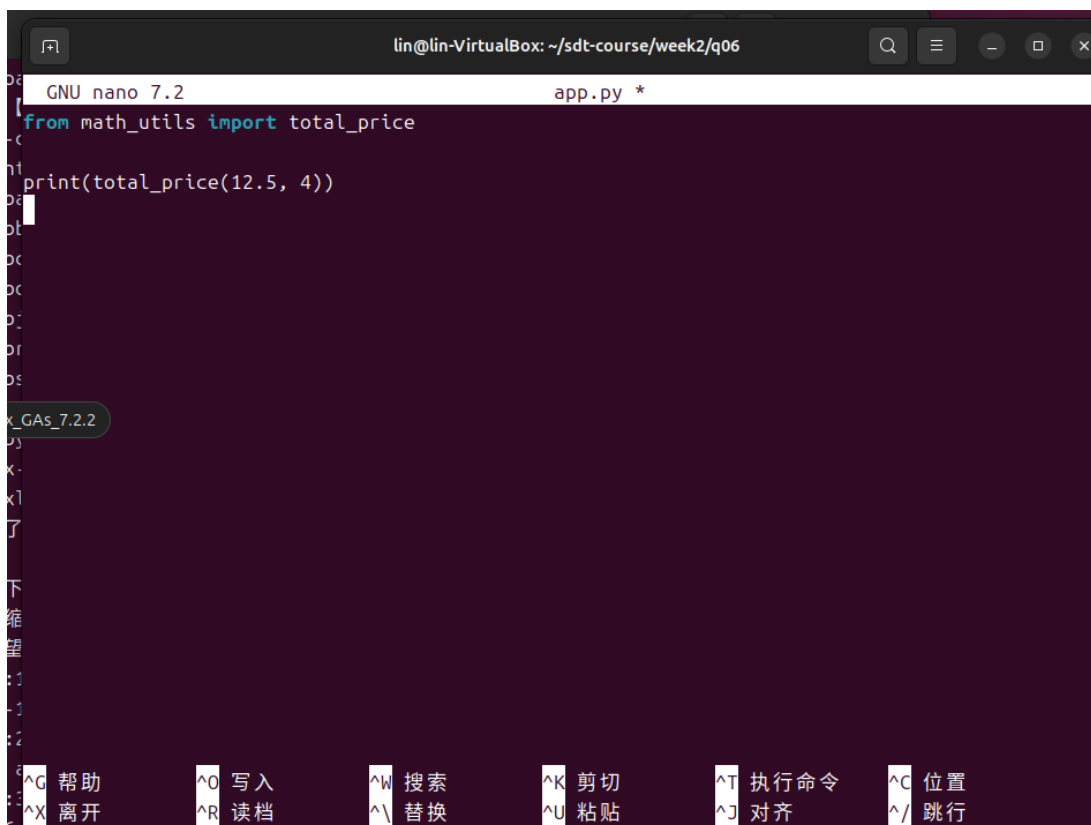
- 问题 1: ruff 检测报 I001 导入块格式错误、F401 导入未使用警告执行 `ruff check` . 检查代码, 报出两类问题: 一是导入语句排版不规范 (I001), 二是 `app.py` 中导入了 `os` 模块但全程没有使用 (F401), 一共 3 处可自动修复的报错。解决: 直接使用 `ruff check --fix` 命令, `ruff` 自动完成删除未使用导入、整理导入语句格式; 修复完成后再次执行 `ruff check .`, 输出 **All checks passed!**, 静态检查全部通过。



```
lin@lin-VirtualBox: ~/sdt-course/week2/q06
GNU nano 7.2 math_utils.py *
def total_price(price: float, count: int) -> float:
    return price * count
```

^G 帮助 ^O 写入 ^W 搜索 ^K 剪切 ^T 执行命令 ^C 位置
^X 离开 ^R 读档 ^\ 替换 ^U 粘贴 ^J 对齐 ^/ 跳行

图 2: q06math_utils.py 截图

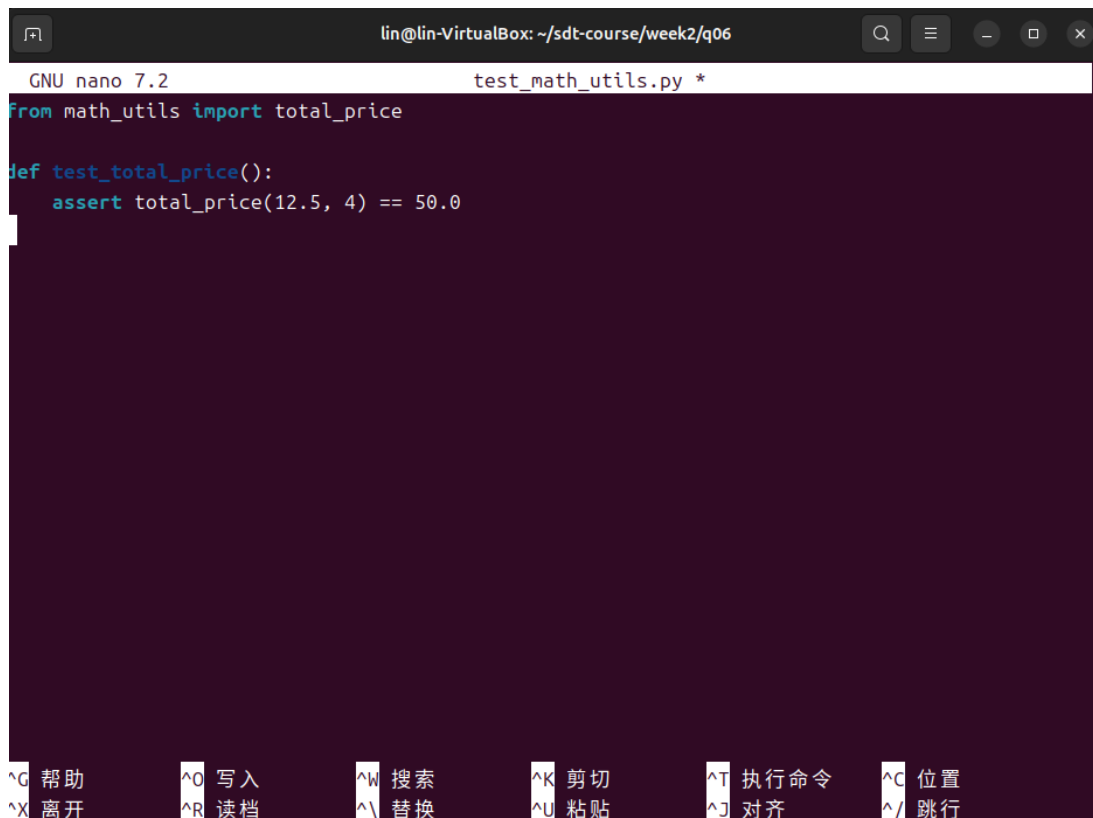


```
lin@lin-VirtualBox: ~/sdt-course/week2/q06
GNU nano 7.2 app.py *
from math_utils import total_price
print(total_price(12.5, 4))
```

x_GAs_7.2.2

^G 帮助 ^O 写入 ^W 搜索 ^K 剪切 ^T 执行命令 ^C 位置
^X 离开 ^R 读档 ^\ 替换 ^U 粘贴 ^J 对齐 ^/ 跳行

图 3: q06app.py 截图

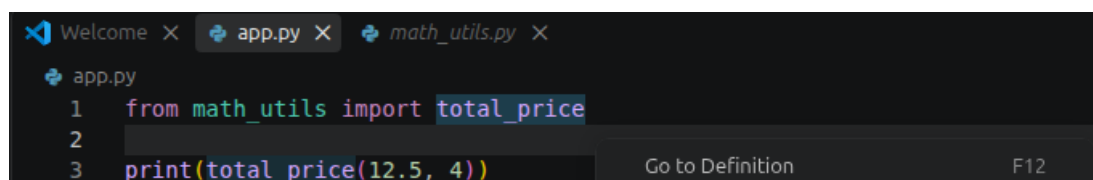


```
lin@lin-VirtualBox: ~/sdt-course/week2/q06
GNU nano 7.2 test_math_utils.py *
from math_utils import total_price

def test_total_price():
    assert total_price(12.5, 4) == 50.0
```

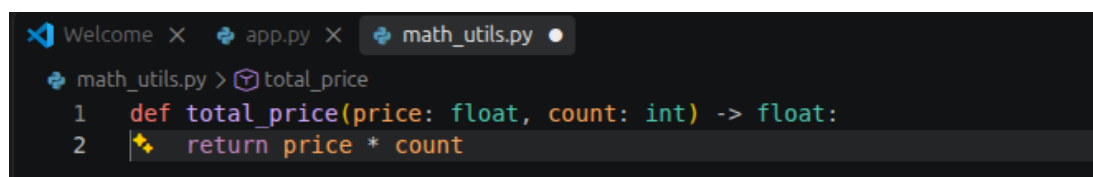
帮助 写入 搜索 剪切 执行命令 位置
离开 读档 替换 粘贴 对齐 跳行

图 4: q06test_math_utils.py 截图



```
Welcome x app.py x math_utils.py x
app.py
1 from math_utils import total_price
2
3 print(total_price(12.5, 4)) Go to Definition F12
```

图 5: q06vscode 跳转定义操作截图



```
Welcome x app.py x math_utils.py
math_utils.py > total_price
1 def total_price(price: float, count: int) -> float:
2     return price * count
```

图 6: q06vscode 跳转定义结果截图

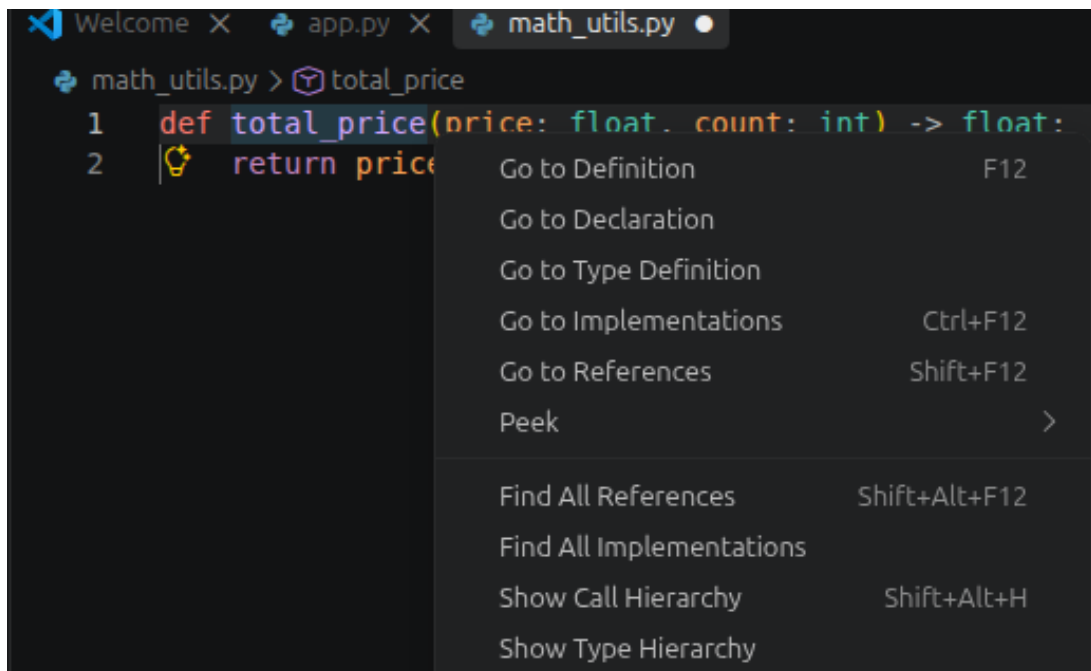


图 7: q06vscode 查找所有引用操作截图

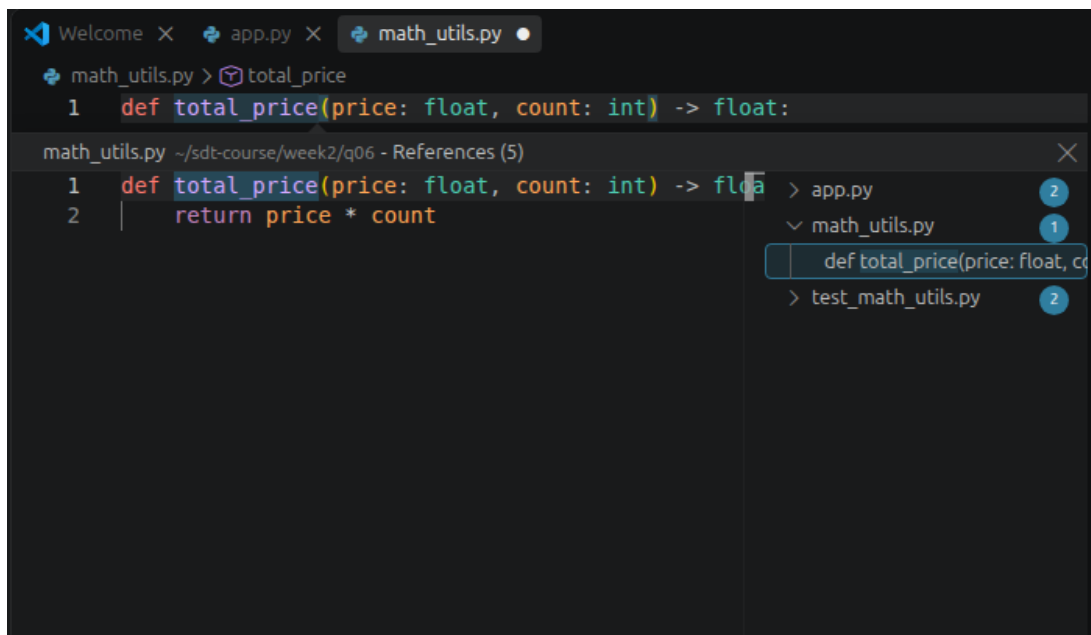


图 8: q06vscode 查找所有引用结果截图

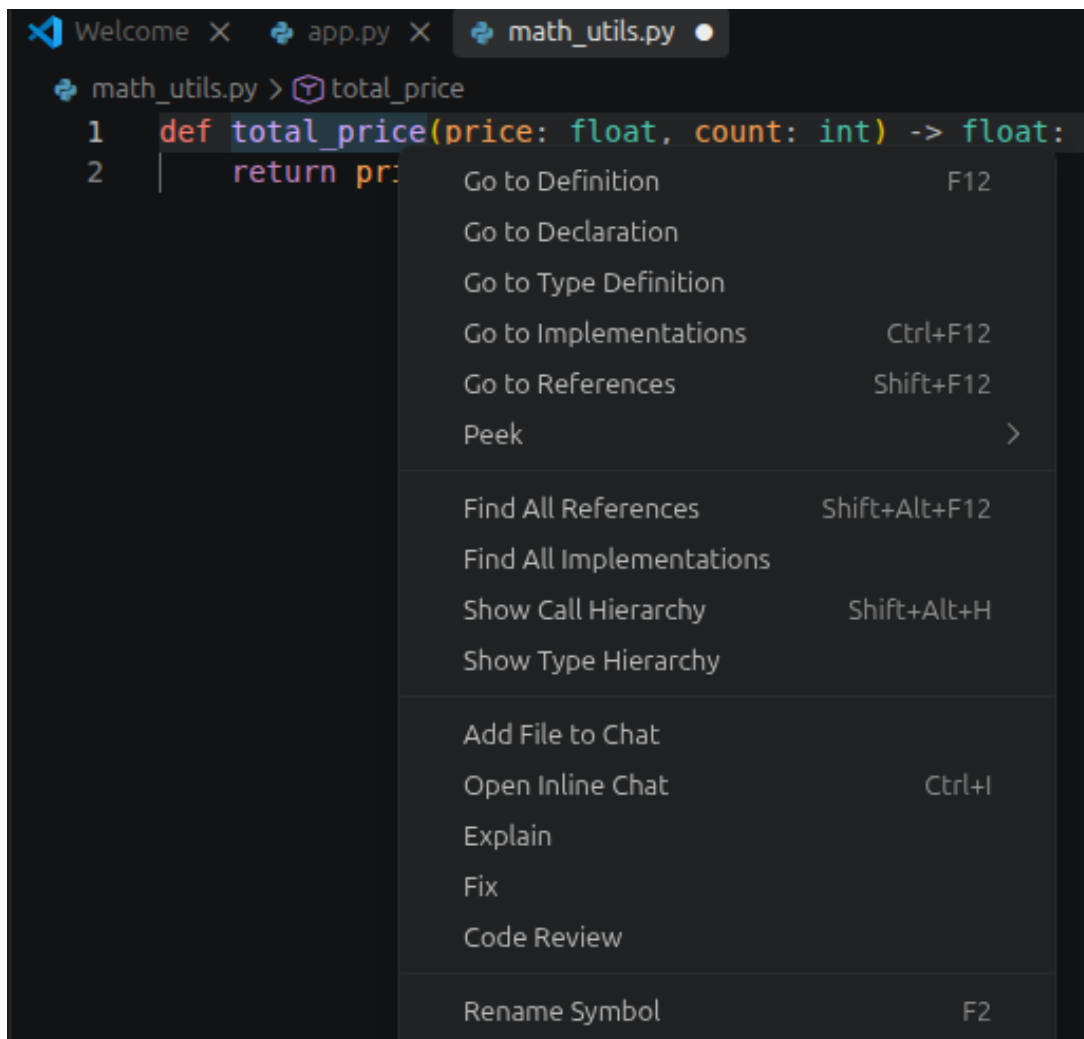


图 9: q06vscode 重命名操作截图 1

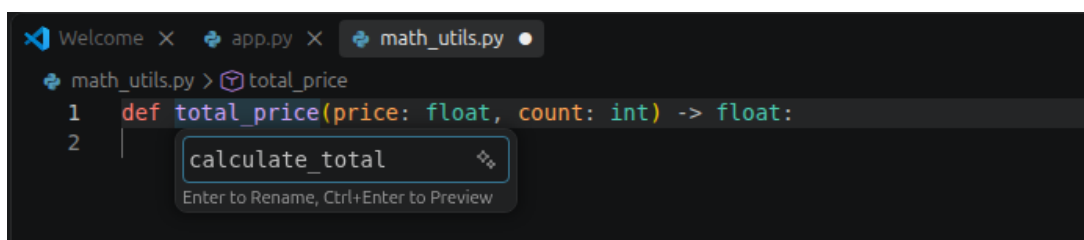


图 10: q06vscode 重命名操作截图 2

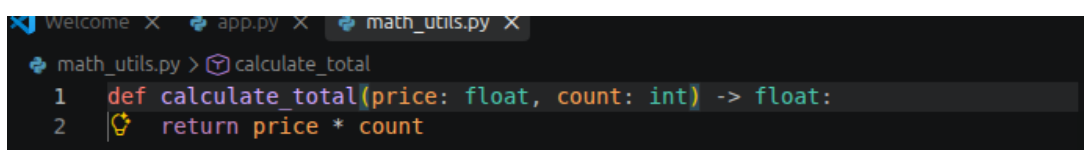


图 11: q06vscode 重命名结果截图 1

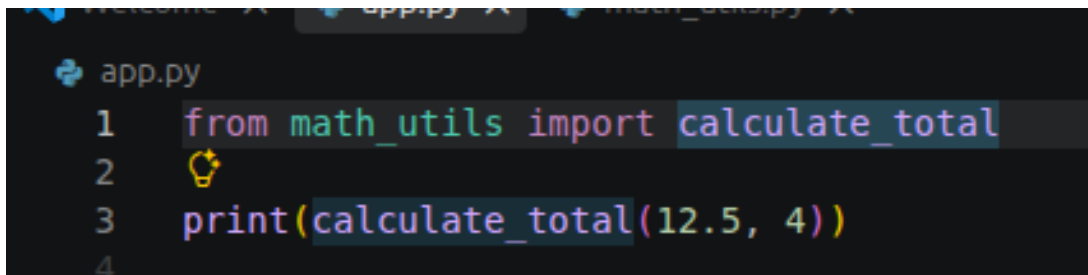


图 12: q06vscode 重命名结果截图 2

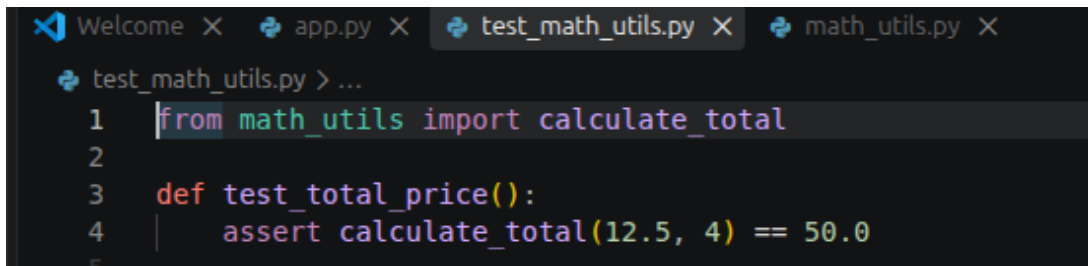


图 13: q06vscode 重命名结果截图 3

2.3 q07: 用调试器定位归并排序缺陷

2.3.1 题目要求

运行存在缺陷的 `merge_sort.py`，确认输出结果异常；使用 `pdb` 调试器在 `merge` 函数内部设置断点，观察 `i`、`j` 下标变量变化；修复归并逻辑缺陷；编写 `pytest` 测试用例，覆盖普通列表与含重复元素的列表，保证测试全部通过，禁止直接调用 `sorted` 替换归并排序逻辑。

2.3.2 实现过程与关键命令

```
nano merge_sort.py
[language=Python]
def merge(left, right):
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i]); i += 1
        else:
            return result + left[i:] + right[j:]
        result.append(right[j]); j += 1 # 此处存在缺陷
def merge_sort(a):
    if len(a) <= 1: return a
    m = len(a) // 2
```

```
lin@lin-VirtualBox:~/sdt-course/week2/q06$ ruff check .
I001 [*] Import block is un-sorted or un-formatted
--> app.py:1:1
|
1 | / import os
2 | | from math_utils import calculate_total
  | | _____^
3 |
4 |     print(calculate_total(12.5, 4))
  |
help: Organize imports
|
1 | import os
2 +
3 | from math_utils import calculate_total
  |

F401 [*] `os` imported but unused
--> app.py:1:8
|
1 | import os
  |      ^^
2 | from math_utils import calculate_total
  |
help: Remove unused import: `os`
|
- import os
1 | from math_utils import calculate_total
  |

I001 [*] Import block is un-sorted or un-formatted
--> test_math_utils.py:1:1
|
1 | from math_utils import calculate_total
  | ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
2 |
3 | def test_total_price():
  |
help: Organize imports
|
2 |
3 +
4 | def test_total_price():
```

```

help: Organize imports
|
2 |
3 +
4 | def test_total_price():
|

Found 3 errors.
[*] 3 fixable with the '--fix' option.
lin@lin-VirtualBox:~/sdt-course/week2/q06$ ruff check --fix
Found 2 errors (2 fixed, 0 remaining).
lin@lin-VirtualBox:~/sdt-course/week2/q06$ ruff check .
All checks passed!
lin@lin-VirtualBox:~/sdt-course/week2/q06$ pytest test_math_utils.py
===== test session starts =====
platform linux -- Python 3.12.3, pytest-9.1.1, pluggy-1.6.0
rootdir: /home/lin/sdt-course/week2/q06
collected 1 item

test_math_utils.py . [100%]

===== 1 passed in 0.00s =====

```

图 15: q06 运行截图 2

```

return merge(merge_sort(a[:m]), merge_sort(a[m:]))

print(merge_sort([3, 1, 4, 1, 5, 9, 2, 6]))

python3 merge_sort.py

python3 -m pdb merge_sort.py
> /home/lin/sdt-course/week2/q07/merge_sort.py(1)<module>()
-> def merge(left, right):
(Pdb) b 4
Breakpoint 1 at /home/lin/sdt-course/week2/q07/merge_sort.py:4
(Pdb) c
> /home/lin/sdt-course/week2/q07/merge_sort.py(4)merge()
-> while i < len(left) and j < len(right):
(Pdb) p i
0
(Pdb) p j
0
(Pdb) p left
[3]
(Pdb) p right
[1]
(Pdb) p left[i]
3
(Pdb) p right[j]
1
(Pdb) n
> /home/lin/sdt-course/week2/q07/merge_sort.py(5)merge()
-> if left[i] <= right[j]:
(Pdb) n

```

```
> /home/lin/sdt-course/week2/q07/merge_sort.py(8)merge()
-> return result + left[i:] + right[j:]
(Pdb) n
--Return--
#进入 else 分支, 直接执行 return, 函数直接返回结束
> /home/lin/sdt-course/week2/q07/merge_sort.py(8)merge()->[3, 1]
-> return result + left[i:] + right[j:]
(Pdb) q

nano merge_sort.py
[language=Python]
def merge(left, right):
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1
    result += left[i:]
    result += right[j:]
    return result

def merge_sort(a):
    if len(a) <= 1:
        return a
    m = len(a) // 2
    return merge(merge_sort(a[:m]), merge_sort(a[m:]))

print(merge_sort([3, 1, 4, 1, 5, 9, 2, 6]))
```

```
python3 merge_sort.py
nano test_merge.py
pytest test_merge.py -v
```

2.3.3 运行结果

修复后排序输出正确；两个 **pytest** 测试用例全部执行通过。

```
Traceback (most recent call last):
File "/home/lin/sdt-course/week2/q07/merge_sort.py", line 17, in <
    module>
print(merge_sort([3, 1, 4, 1, 5, 9, 2, 6]))
```

```

~~~~~
File "/home/lin/sdt-course/week2/q07/merge_sort.py", line 15, in
    merge_sort
return merge(merge_sort(a[:m]), merge_sort(a[m:]))
~~~~~

File "/home/lin/sdt-course/week2/q07/merge_sort.py", line 15, in
    merge_sort
return merge(merge_sort(a[:m]), merge_sort(a[m:]))
~~~~~

File "/home/lin/sdt-course/week2/q07/merge_sort.py", line 4, in merge
while i < len(left) and j < len(right):
~~~~~

TypeError: object of type 'NoneType' has no len()

[1, 1, 2, 3, 4, 5, 6, 9]

===== test session starts
=====
platform linux -- Python 3.12.3, pytest-9.1.1, pluggy-1.6.0 -- /usr/bin
/python3
cachedir: .pytest_cache
rootdir: /home/lin/sdt-course/week2/q07
collected 2 items

test_merge.py::test_case1 PASSED                                [ 50%]

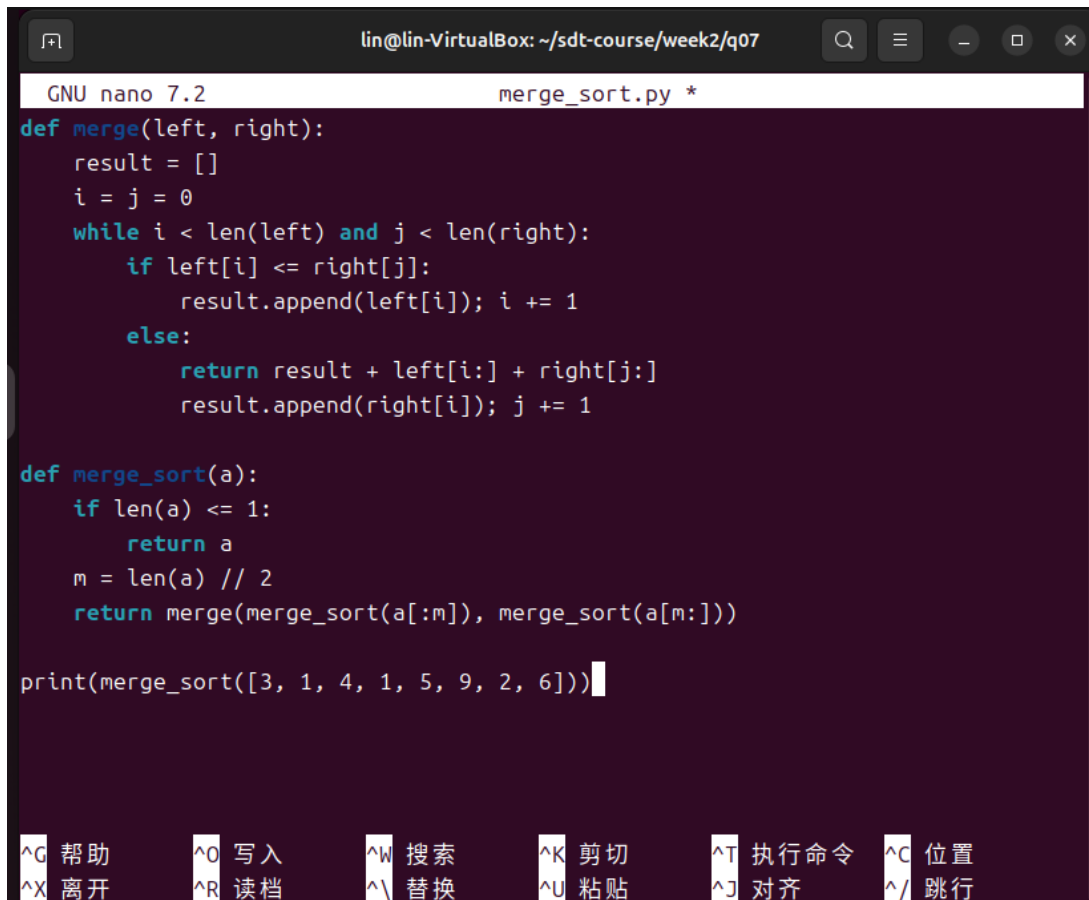
test_merge.py::test_dup PASSED                                  [100%]

===== 2 passed in 0.01s
=====

```

2.3.4 遇到的问题与解决

- 问题 1: `pdb` 调试发现原始代码在 `else` 分支直接执行 `return`, 归并循环提前终止, 数组拼接不完整, 排序结果错乱。解决: 删除错误的提前 `return` 语句, 完成 `while` 循环后再拼接剩余切片, 最后返回合并结果。



The screenshot shows a terminal window with the title bar "lin@lin-VirtualBox: ~/sdt-course/week2/q07". The editor is GNU nano 7.2, editing merge_sort.py. The code is as follows:

```
def merge(left, right):
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i]); i += 1
        else:
            return result + left[i:] + right[j:]
            result.append(right[i]); j += 1

def merge_sort(a):
    if len(a) <= 1:
        return a
    m = len(a) // 2
    return merge(merge_sort(a[:m]), merge_sort(a[m:]))

print(merge_sort([3, 1, 4, 1, 5, 9, 2, 6]))
```

The bottom of the window shows nano editor shortcuts: ^G 帮助, ^O 写入, ^W 搜索, ^K 剪切, ^T 执行命令, ^C 位置, ^X 离开, ^R 读档, ^\ 替换, ^U 粘贴, ^J 对齐, ^/ 跳行.

图 16: q07merge_sort.py 修改前截图



The screenshot shows the same terminal window as Figure 16, but with the code modified. The changes are highlighted in red in the original image:

```
def merge(left, right):
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1
        # return移到循环外面
        return result + left[i:] + right[j:]

def merge_sort(a):
    if len(a) <= 1:
        return a
    m = len(a) // 2
    return merge(merge_sort(a[:m]), merge_sort(a[m:]))

print(merge_sort([3, 1, 4, 1, 5, 9, 2, 6]))
```

图 17: q07merge_sort.py 修改后截图

```
GNU nano 7.2                                test_merge.py *
from merge_sort import merge_sort

def test_case1():
    #题目给的输入
    assert merge_sort([3, 1, 4, 1, 5, 9, 2, 6]) == [1,1,2,3,4,5,6,9]

def test_dup():
    #含重复元素列表
    assert merge_sort([5,2,2,8,2,1]) == [1,2,2,2,5,8]
```

图 18: q07test_merge.py 截图

```
lin@lin-VirtualBox: ~/sdt-course/week2/q07
lin@lin-VirtualBox:~$ cd ~/sdt-course/week2
lin@lin-VirtualBox:~/sdt-course/week2$ mkdir q07 && cd q07
lin@lin-VirtualBox:~/sdt-course/week2/q07$ nano merge_sort.py
```

图 19: q07 运行截图 1

```
lin@lin-VirtualBox:~/sdt-course/week2/q07$ python3 merge_sort.py
Traceback (most recent call last):
  File "/home/lin/sdt-course/week2/q07/merge_sort.py", line 17, in <module>
    print(merge_sort([3, 1, 4, 1, 5, 9, 2, 6]))
    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/lin/sdt-course/week2/q07/merge_sort.py", line 15, in merge_sort
    return merge(merge_sort(a[:m]), merge_sort(a[m:]))
    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/lin/sdt-course/week2/q07/merge_sort.py", line 15, in merge_sort
    return merge(merge_sort(a[:m]), merge_sort(a[m:]))
    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/lin/sdt-course/week2/q07/merge_sort.py", line 4, in merge
    while i < len(left) and j < len(right):
    ^^^^^^^^^
TypeError: object of type 'NoneType' has no len()
```

图 20: q07 运行截图 2

```

lin@lin-VirtualBox:~/sdt-course/week2/q07$ python3 -m pdb merge_sort.py
> /home/lin/sdt-course/week2/q07/merge_sort.py(1)<module>()
-> def merge(left, right):
(Pdb) b 4
Breakpoint 1 at /home/lin/sdt-course/week2/q07/merge_sort.py:4
(Pdb) c
> /home/lin/sdt-course/week2/q07/merge_sort.py(4)merge()
-> while i < len(left) and j < len(right):
(Pdb) p i
0
(Pdb) p j
0
(Pdb) p left
[3]
(Pdb) p right
[1]
(Pdb) p left[i]
3
(Pdb) p right[j]
1
(Pdb) n
> /home/lin/sdt-course/week2/q07/merge_sort.py(5)merge()
-> if left[i] <= right[j]:
(Pdb) n
> /home/lin/sdt-course/week2/q07/merge_sort.py(8)merge()
-> return result + left[i:] + right[j:]
(Pdb) n
--Return--
> /home/lin/sdt-course/week2/q07/merge_sort.py(8)merge()->[3, 1]
-> return result + left[i:] + right[j:]

```

图 21: q07 运行截图 3

```

(Pdb) n
> /home/lin/sdt-course/week2/q07/merge_sort.py(8)merge()
-> return result + left[i:] + right[j:]
(Pdb) n
--Return--
> /home/lin/sdt-course/week2/q07/merge_sort.py(8)merge()->[4, 1]
-> return result + left[i:] + right[j:]
(Pdb) q
lin@lin-VirtualBox:~/sdt-course/week2/q07$ nano merge_sort.py
lin@lin-VirtualBox:~/sdt-course/week2/q07$ python3 merge_sort.py
lin@lin-VirtualBox:~/sdt-course/week2/q07$ python3 merge_sort.py
lin@lin-VirtualBox:~/sdt-course/week2/q07$ nano merge_sort.py
lin@lin-VirtualBox:~/sdt-course/week2/q07$ python3 merge_sort.py
[1, 1, 2, 3, 4, 5, 6, 9]

```

图 22: q07 运行截图 4

```
lin@lin-VirtualBox:~/sdt-course/week2/q07$ nano test_merge.py
lin@lin-VirtualBox:~/sdt-course/week2/q07$ pytest test_merge.py -v
===== test session starts =====
platform linux -- Python 3.12.3, pytest-9.1.1, pluggy-1.6.0 -- /usr/bin/python3
cachedir: .pytest_cache
rootdir: /home/lin/sdt-course/week2/q07
collected 2 items

test_merge.py::test_case1 PASSED [ 50%]
test_merge.py::test_dup PASSED [100%]

===== 2 passed in 0.01s =====
```

图 23: q07 运行截图 5

2.4 q08: 先测量，再优化慢速词频程序

2.4.1 题目要求

运行 `generate_words.py` 生成 `words.txt` 测试数据；使用 `time` 工具统计原始程序两次运行耗时；使用 `cProfile` 做性能分析定位热点；在不写死数量常量、不改变输出 `count` 结果的前提下优化去重逻辑；对比优化前后的运行耗时与加速比。

2.4.2 实现过程与关键命令

```
cd ~/sdt-course/week2/q08
nano generate_words.py
nano wordfreq.py
python3 generate_words.py
time python3 wordfreq.py
time python3 wordfreq.py
python3 -m cProfile -s cumulative wordfreq.py
nano wordfreq.py
time python3 wordfreq.py
time python3 wordfreq.py
```

2.4.3 运行结果

原始版本使用 `list` 的 `in` 操作做去重，时间复杂度较高；替换为 `set` 集合进行去重后，程序运行速度得到明显提升。原始程序与优化后程序输出计数均为 `count= 952`，输出结果保持一致。

- 优化前两次实测 **real** 时间：0.021 s、0.023 s，中位数 0.022 s。
- 优化后两次实测 **real** 时间：0.018 s、0.013 s，中位数 0.0155 s。
- 基于中位数计算得到加速比约为 1.42。

```

count= 952

real    0m0.021s
user    0m0.016s
sys     0m0.005s

count= 952

real    0m0.023s
user    0m0.022s
sys     0m0.000s

count= 952
964 function calls in 0.009 seconds

Ordered by: cumulative time

ncalls tottime percall cumtime percall filename:lineno(function)
1      0.000    0.000    0.009    0.009 {built-in method builtins.exec}
1      0.008    0.008    0.009    0.009 wordfreq.py:1(<module>)
1      0.000    0.000    0.000    0.000 {built-in method builtins.print}
1      0.000    0.000    0.000    0.000 {method 'split' of 'str' objects}
952    0.000    0.000    0.000    0.000 {method 'append' of 'list' objects}
1      0.000    0.000    0.000    0.000 {built-in method _io.open}
1      0.000    0.000    0.000    0.000 {method 'disable' of '_lsprof.Profiler
' objects}
1      0.000    0.000    0.000    0.000 {method 'read' of '_io.TextIOWrapper'
objects}
1      0.000    0.000    0.000    0.000 <frozen codecs>:319(decode)
1      0.000    0.000    0.000    0.000 <frozen codecs>:309(__init__)
1      0.000    0.000    0.000    0.000 {built-in method _codecs.utf_8_decode}
1      0.000    0.000    0.000    0.000 {built-in method builtins.len}
1      0.000    0.000    0.000    0.000 <frozen codecs>:260(__init__)

count= 952

real    0m0.018s
user    0m0.005s
sys     0m0.010s

count= 952

real    0m0.013s
user    0m0.007s

```

sys 0m0.003s

```
GNU nano 7.2 generate_words.py *
import random
random.seed(20260831)
vocab = [f"w{i}" for i in range(1000)]
with open("words.txt", "w", encoding="utf-8") as f:
    f.write(" ".join(random.choice(vocab) for _ in range(3000)))
```

图 24: q08generate_words.py 截图

```
lin@lin-VirtualBox: ~/sdt-course/week2/q08
GNU nano 7.2 wordfreq.py *
words = open("words.txt", encoding="utf-8").read().split()
unique = []
for word in words:
    if word not in unique:
        unique.append(word)
print("count=", len(unique))
```

图 25: q08wordfreq.py 优化前截图

```
GNU nano 7.2 wordfreq.py *
words = open("words.txt", encoding="utf-8").read().split()
unique = set(words)
print("count=", len(unique))
```

图 26: q08wordfreq.py 优化后截图

2.4.4 遇到的问题与解决

- 问题 1: 原始版本使用 `list` 的 `in` 操作做去重, 程序运行耗时很长。原始代码把新元素不断 `append` 进 `list`, 再用 `if word not in unique` 判断; `list` 成员查找时间复杂度为 $O(n)$, 数据量大的时候整体效率很低, 两次 `time` 测量可以观察到明显的耗时。解决: 改用 `set` 集合存储已经出现过的单词, 集合成员查询为 $O(1)$, 在不修改输出 `count` 结果、不写死 `1000` 这个常量的前提下完成优化。
- 问题 2: 优化前后计时只运行一次, 偶然因素干扰大, 不方便计算中位数与加速比。单次 `time` 测量会受系统后台进程、虚拟机负载波动影响, 单次结果不具备参考意义。解决: 题目要求原程序、优化后程序分别运行 2 次, 记录每组两次耗时, 取中位数, 再计算优化后的加速比。

```

lin@lin-VirtualBox:~$ cd ~/sdt-course/week2/q08
lin@lin-VirtualBox:~/sdt-course/week2/q08$ nano generate_words.py
lin@lin-VirtualBox:~/sdt-course/week2/q08$ nano wordfreq.py
lin@lin-VirtualBox:~/sdt-course/week2/q08$ python3 generate_words.py
lin@lin-VirtualBox:~/sdt-course/week2/q08$ time python3 wordfreq.py
count= 952

real    0m0.021s
user    0m0.016s
sys     0m0.005s
lin@lin-VirtualBox:~/sdt-course/week2/q08$ time python3 wordfreq.py
count= 952

real    0m0.023s
user    0m0.022s
sys     0m0.000s
lin@lin-VirtualBox:~/sdt-course/week2/q08$ python3 -m cProfile -s cumulative wordfreq.py
count= 952
      964 function calls in 0.009 seconds

Ordered by: cumulative time

ncalls  tottime  percall  cumtime  percall filename:lineno(function)
      1   0.000   0.000   0.009   0.009 {built-in method builtins.exec}
      1   0.008   0.008   0.009   0.009 wordfreq.py:1(<module>)
      1   0.000   0.000   0.000   0.000 {built-in method builtins.print}
      1   0.000   0.000   0.000   0.000 {method 'split' of 'str' objects}
     952   0.000   0.000   0.000   0.000 {method 'append' of 'list' objects}
      1   0.000   0.000   0.000   0.000 {built-in method _io.open}
      1   0.000   0.000   0.000   0.000 {method 'disable' of '_lsprof.Profiler' objects}
      1   0.000   0.000   0.000   0.000 {method 'read' of '_io.TextIOWrapper' objects}
      1   0.000   0.000   0.000   0.000 <frozen codecs>:319(decode)
      1   0.000   0.000   0.000   0.000 <frozen codecs>:309(__init__)
      1   0.000   0.000   0.000   0.000 {built-in method _codecs.utf_8_decode}
      1   0.000   0.000   0.000   0.000 {built-in method builtins.len}
      1   0.000   0.000   0.000   0.000 <frozen codecs>:260(__init__)

```

图 27: q08 运行截图 1

```

lin@lin-VirtualBox:~/sdt-course/week2/q08$ nano wordfreq.py
lin@lin-VirtualBox:~/sdt-course/week2/q08$ time python3 wordfreq.py
count= 952

real    0m0.018s
user    0m0.005s
sys     0m0.010s
lin@lin-VirtualBox:~/sdt-course/week2/q08$ time python3 wordfreq.py
count= 952

real    0m0.013s
user    0m0.007s
sys     0m0.003s

```

图 28: q08 运行截图 2

3 课后练习

实例 1: 进入 `./ssh/`, 检查你是否已经有一对 SSH 密钥。如果没有, 就用 `ssh-keygen -a 100 -t ed25519` 生成一对。

命令:

```
cd ~/.ssh
ls -la
ssh-keygen -a 100 -t ed25519
```

输出: 生成密钥对。

实例 2: 编辑 `ssh/config`, 加入 Host `vm` 配置项。

命令:

```
nano ~/.ssh/config
```

写入内容:

```
Host vm
User username_goes_here
HostName ip_goes_here
IdentityFile ~/.ssh/id_ed25519
LocalForward 9999 localhost:8888
```

输出: 保存退出, 配置文件写入完成。

实例 3: 使用 `ssh-copy-id vm` 把你的 SSH 公钥复制到服务器上。

命令:

```
ssh-copy-id vm
```

输出: 提示输入虚拟机用户密码, 完成公钥推送; 后续 `ssh vm` 登录不再需要输入密码。

实例 4: 在虚拟机内启动 `http.server 8888`, 利用本地端口转发访问服务。

命令:

```
# 虚拟机内
python3 -m http.server 8888
```

输出: 在本机浏览器访问 `http://localhost:9999`, 可以正常看到 **web** 服务页面。

实例 5: 可执行斐波那契代码, 运行程序。

命令:

```
chmod +x fib_exercise.py
python3 fib_exercise.py
```

输出: 34

实例 6: 使用 `journalctl` 获取最近一天 `sudo` 相关日志。

命令:

```
sudo ls
journalctl --since "1 day ago" | grep -E 'sudo|USER=root'
```

输出: 打印出最近一天 **sudo** 执行操作的系统日志, 可以看到刚刚执行 **sudo ls** 的相关日志记录。

实例 7: 浏览 IDE 扩展列表, 安装有用的 VS Code 扩展。

命令: VS Code 打开扩展面板, 搜索并安装扩展。

输出: 扩展列表中出现已安装扩展。

实例 8: 启动 **http.server 4444**, **lsof** 查找监听端口, **kill** 终止进程。

命令:

```
# 终端A
python3 -m http.server 4444
# 终端B
lsof | grep LISTEN
kill <PID>
```

输出: **lsof** 输出可以看到 **python** 进程监听 **4444** 端口; 执行 **kill** 之后, 终端 A 的 **web** 服务进程终止。

实例 9: **stress -c 3** 与 **taskset -cpu-list 0,2** 绑定 CPU, **htop** 可视化。

命令:

```
stress -c 3
htop
taskset --cpu-list 0,2 stress -c 3
htop
```

输出: 第一次 **htop** 观察 3 个 CPU 核心全部占满; **taskset** 绑定后, 压力仅分布在 CPU0、CPU2, 其余核心空闲。

实例 10: **cgroups v2** 限制 **stress -m** 内存使用。

命令:

```
sudo mkdir -p /sys/fs/cgroup/memlimit
echo $((256*1024*1024)) | sudo tee /sys/fs/cgroup/memlimit/memory.max
stress -m 1
pidof stress
echo <PID> | sudo tee /sys/fs/cgroup/memlimit/cgroup.procs
sudo rmdir /sys/fs/cgroup/memlimit
```

输出: **stress** 进程内存被限制最大 256MB, 可读取 **cgroup** 文件观察内存统计, 实验结束删除 **cgroup** 目录。

4 解题感悟

第二周实验主要学习 **Linux** 进程生命周期、信号处理、任务后台调度、调试器排错、性能分析、CPU 亲和性、**cgroups v2** 内核资源隔离与 **ulimit** 会话级资源限制。通过多 **SSH** 终端窗口，模拟了真实运维场景：一个终端运行被测程序，其他终端完成观测、信号发送、资源监控。

实验过程体会到老旧开源软件在新版本 **Python** 环境会出现兼容性问题，遇到安装报错不应当一味添加破坏系统的参数，可以优先在文档记录现象，保留源码与运行结果完成作业。**cgroups v2** 作为内核伪文件系统，拥有严格接口约束，**cgroup.procs** 仅支持单行单个 **PID**；**sysfs** 下配置全部驻留内存，虚拟机关机重启后全部丢失；子目录仅能使用 **rmdir** 删除。**ulimit** 仅对当前 **shell** 会话以及子进程生效，不能实现系统全局永久修改。

调试器 **pdb**、性能分析工具 **cProfile** 可以帮助定位程序逻辑缺陷与性能瓶颈；**IDE** 语义重构工具可以安全完成标识符全局修改，但需要等待语言服务器索引完成，否则会出现修改不全。课程强调可复现、可记录的实操习惯，每完成一部分实验做好截图记录；使用 **git** 进行版本管理，合理配置 **.gitignore**，不对虚拟环境、缓存临时文件进行版本提交，保留完整的作业版本历史。

5 代码仓库提交记录

GitHub 仓库链接：<https://github.com/imnotab/sdtb/tree/main>

提交历史截图：

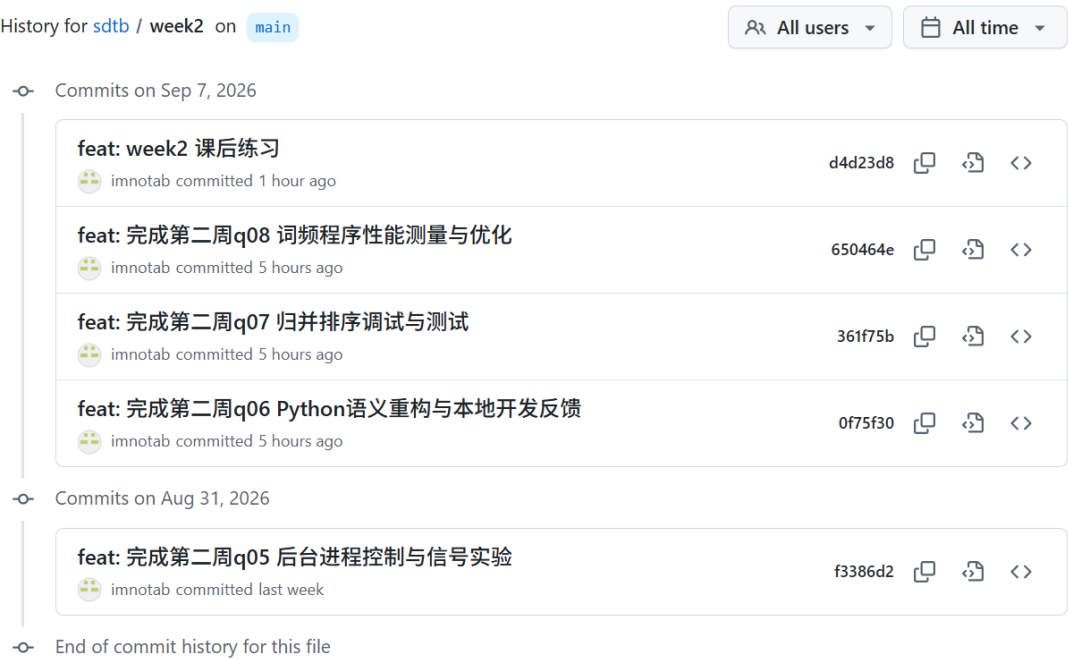


图 29: 项目 commit 历史记录